Dt. _____ B+
Pg. _____

source code → machine code

Binary (0, 1)

Interpreter	Compiler
1) Terminate where ever	1) Not terminate but compile all errors and show at end
2) Directly output	2) Firstly generate code then run
3) Not dependent on 3rd party.	3) Third party dependent
eg Washing machine → Output	eg Washing machine → Microprocessor → output

Low-level language	High-level
1) Hardware dependent	Hardware independent
eg Assembly, Fortran	eg C++, Java
eg: MOV A, 80H (Full address)	int a

1) Keywords
2) Variables or Identifiers:
   (i) A keyword can't be used as Variable
   (ii) It can be alphanumeric, it cannot start with number
   (iii) Space is not allowed
   (iv) Only one special character (underscore) allowed

Datatype (1) char (2) int (3) float

(1) char: 1 byte = 8 bits = $2^7 - 2^0$ = 0 to 255

char section = 'A'; is similar to char section = 65;

A → 65
a → 97
0 → 48

Be Positive...

Scanned with CamScanner

Pg. B+

(2) int: 2 bytes / 4 bytes ↘ 64 processor
32 processor
16 bits → 2^{15}
32 bits → 2^{31}
int a = 7142 (decimal)
2^{31} : 0 to 9

(3) float: 4 bytes
float a = 31.42.

Operators

(1) Assignment: =
a = b
a = 10
10 = a ✗
a = b = c = 0
a = (10, 20, 30)
a = (10, 20, 30)

(2) Arithmetic
+, −, *, /, (%) → not used in float

cin >> a, b; ✗
cin >> (a), (b); ✗

10→ Enter two values and swap with and without using 3rd variable
20→ Enter marks in 5 subjects and display average
30→ Enter seconds, convert to hrs and minutes
40→ Enter 3 digits # and display its digits separately

Scanned with CamScanner

→ int a=11, b=3;
float c;
c = a/b; = 3.000000
↓
c = (float)a/b; = 3.666666
explicit cast

→ int a=10
float b=20; → implicit cast, value in b
b = a; // 10.000000. 4 byte can be in 2
a = b; // X ∵ 2 byte not stored in 4

1) int main()
{ int a, b, t;
t = a;
a = b;
b = t;
}

int main()
{ int a, b;
a = a+b;
b = a-b;
a = a-b;
}

2) int main()
{
float Eng, Phy, Chem; Average = 0;
cout << "Enter marks in Eng, Phy, Chem";
cin >> Eng >> Phy >> Chem;
Average = (Eng+Phy+Chem) / 3.0;
Cout << Average << endl;
}

Scanned with CamScanner

3)
```
int main()
{ int seconds, hrs, min;
cout<<"Enter seconds";
cin>>seconds;
min = seconds / 60;
hrs = min / 60;
hrs = hrs + (min%60);
min = min%60;
seconds = seconds%60;
cout<<hrs<<" "<<min<<" "<<seconds;
return 0;
}
```

4)
```
int main()
{ int a,b,c,d;
cout<<"Enter three digit number"<<endl;
cin>>a;
for(int i=0; i<3; i++)
b = a%10;
a = ((b-(a%10))/10)%10
b = a%10;
c = (a/10)%10;
d = (a/100)%10;
cout<<"Unit digit="<<b<<endl;
cout<<"Tens digit="<<c<<endl;
cout<<"Hundreds digit="<<d<<endl;
```

or use

```
and
for(i=0;i<3;i++)
b = a%10;
a = a/10;
b = b*10+a;
```

→ Basic Structure ⇒

① import or include useful files from library
② Specify the start pt for execution

```
#include <iostream>
        ↳ Header file   input output stream
main()
{
}
```

mandatory & only one

only req if U want to cin or cout

iostream → iostream.h → Turbo C++ online

hexadecimal form (To burn file)

already executed

cin
cout

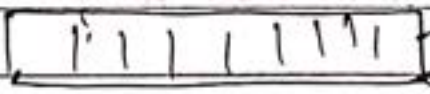
std::cin
std::cout

int main( ) in code blocks

{ return 0; / 10; / 20; } → Successful compilation

(use any)

iostream

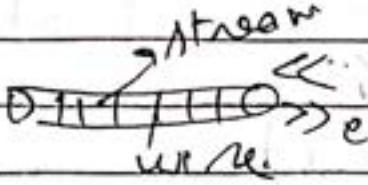


iostream (2 classes)

istream (cin)

ostream (2 classes)

cout (objects)



Scanned with CamScanner

Dt. _____
Pg. _____ B+

iomanip
clrscr()
getch()

Control flow Statements

- Decision Making: if, if else, Nested, Ladder if, Switch
- Looping: do while, while, for
- Jump: break, continue

operator → Relational >, <, >=, <=, !=, ==

→ Logical &&, ||, !

→ Bitwise &, |, ~ NOT, ^ XOR, >> Shift, << Shift

→ Conditional

Notes → 1) we may skip the blocks if ∄ a single statement inside if or else
2) else must be present immediate after if

1)
```
int a=1, b=1;
if(a>b)
  a++;
  b++;
cout<<a<<b;
```
output: 1 2

2)
```
int a=5, b=6;
if(!(a>b))
  a++;
  b++;
cout<<a<<b;
```
output: 6 7

Dt. _____ B+
Pg. _____

8) int a=5, b=6;
if (a>b)
a++;
b++;
else
cout<<a<<b;

Error

dangling else → when if confuses in nested if else ca

ladder if:
if (condition 1)
task 1;
else if (condition 2)
task 2;
else if (condition 3)
task 3;
else
taskn;

10 → Enter three values and display largest

20 → Enter price of a product and provide discount as 5% if price is upto 10,000, 10% if price upto 20,000 15% if price upto 30,000 and 25% if price > 30,000?
Display actual amount, discount, final amount

30 → Enter two time formats

h	m	s
8	40	30
6	30	40

and add them in 12 hours format

1) int main()
int a, b, c; cout<<"Enter 3 numbers";
cin>>a>>b>>c;
if (a>b)
{

Be Positive...

Scanned with CamScanner

int max = (a>c)? a:c;
}
else
{
int max = (b>c)? b:c;
} cout << "largest of 3 is = " << max;
return 0;
}

(2)
int main()
{
int price, discount, np;
cout << "Enter price" << endl;
cin >> price;
if (price <= 10,000)
{ discount = 5/100
np = price * discount }
if (~~price~~ price > 10,000 && price <= 20,000)
{
ducount = 10/100
np = price * discount }
if (price > 20,000 && price <= 30,000)
~~np = price * discount~~
discount = 15/100
np = price * discount }
else
{ discount = 25/100
np = price * discount
}

Scanned with CamScanner

cout << "Old price is = " << price << endl;
cout << "Discount is = " << Discount << endl;
cout << "newprice is = " << np << endl;
return 0;
}

2) int t1, t2;
cout << "enter two times t1, t2" << endl;
cin
cout <<

3) int h1, h2, m1, m2, s1, s2;
cout << "Enter hours of two times h1, h2" << endl;
cin >> h1 >> h2;
cout << "Enter min of two times m1, m2" << endl;
cin >> m1 >> m2;
cout << "Enter seconds of two times s1, s2" << endl;
cin >> s1 >> s2;

int s = (s1 + s2) % 60;
int m = (m1 + m2) % 60

int s = (s1 + s2) / 60;
int m = ((m1 + m2) + s) / 60;
int h = (h1 + h2 + m) % 10;
s = s % 60; int sec = (s1 + s2) % 60
m = m % 60; int min = (m1 + m2 + s) % 60;
cout << "Time after adding two times is :
cout << h << " hours" << min << "minutes" << sec << "seconds" << endl;
return 0;
}

Be Positive...

Scanned with CamScanner

```
int val1, val2, choice;
cout << "Enter two values: ";
cin >> val1 >> val2;
cout << "Enter an option: \n1.sum
\n2.sub.\n3.mult\n";
cin >> choice;
switch (choice)
{
case 1:
cout << "sum = " << (a+b);
break;
case 2:
cout << "sub = " << (a-b);
break;
case 3:
cout << "mul = " << (a*b);
break;
default:
cout << "Wrong option";
}
}
```

Note - 1) We can't use relational or logical operator in case of switch
2) No float used in switch

Q-1 Enter a date Suppose today is monday. Which day will occur after n days.

```
int n;
cout << "Enter value of n" << endl;
cin >> n;
cout << "Today is Monday" << endl;
```

Be Positive..

Scanned with CamScanner

Dt. _____
Pg. 11 B+

if ((n%7)==1)
cout << "Day after n days is" << Tuesday << endl;
if (n%7==2)
cout << "Day after n days is" << Wed << endl;
if (n%7==3)
cout << "Day after ——— " << Thu << endl;
if (n%7==4)
cout <<
—
—
return 0;
}

loops → Iterations
Requirements
1) Initialization
2) condition
3) Direction

i) Do-while
do
{
task;
} while(condition);
→ ATMs (application)
execute once atleast

2) While
while (1) → infinite
{ not run
task; if while(0) or while(-1)
}

Scanned with CamScanner

Dt. _____ B+
Pg. _____

Q- Find Sq up to 10 by both loops

→ int a = 1
do
{
cout << a * a << endl; a++
} while (a <= 10)

→ int a = 1
~~for~~ while (a <= 10)
{ cout << a * a << endl;
a++;
}

(3) for (master) master loop :
(1)
for (int j condi; ↑ | ↓)
{ (2)
(3) body
} (4)

~~1 4 9 16 25 36 49 64 81 100~~
1 4 9 16 25 36 49 64 81 100

Alter methods for loop
int a = 10
for (; a <= 20; a++)
or
for (; a <= 20;)
a++;

Scanned with CamScanner

(13) As soon as condition gets false loop ends

```
for(a=1, b=10 ; a<=5 ; a++, b--)
{
cout<<a<<b
}
```

or

```
;a<=5 || b>=5;
```

Q→ Display 30 29 27 24 20 15 9 2

```
for(int i=30 ; i>=2 ; i--)
{
cout<<i;
i=i-1 ;
}
```

```
for(int i=30, int j=1; i>=2 ; i=i-j, j++)
cout<<i;
i=i-j ;
}
```

Q→ Display sum of 1st n natural #
Q→ factorial of given number
Q→ for(n=1; n= Check whether # is prime or not
Q→ Display the table of given #
Q→ Count the digits in a # eg 1221
Q→ Reverse a # palindrome same
Q→ Check whether # is ~~palindrome~~ or not

Pg. B+

a → 3246 — Rev

/10 → 6 — %10 → 6

/10 → 324 → 4 — %10 → ×10 → 64

/10 → 32 → 2 — %10 → ×10 → + 642

/10 → 3 → 3 — %10 → ×10 → + 6423

~~while (a/10 == 0)~~

→ Reverse digits

```
int a, b, rev=0
cout << " Enter a number: ";
cin >> a;
while(a>0)
{ b= a%10;
  rev= rev*10+b;
  a=a/10;
}
cout << "reverse" << rev;
}
```

→ int a = 15;

```
cout << a;          15
cout << ++a;        16
cout << a;          16
cout << a++;        16
cout << a;          17
cout << a++;        17
cout << ++a;        19
cout << a;          19
```

Scanned with CamScanner

Dt.
Pg. 15 B+

int a=1; b=1;
a = a++ + ++b;
b = b++ + ++a;
cout << a << b;

output
a = 5
b = 8

int a=1;
cout << ++a << ++a << ++a;

output
4 4 4

(work right to left
first increment all
then display)

⇒ cout << ++a << a++ << ++a;

output 4 2 4

(first display then increment)

cout << ++a << a++ << a << ++a << a++ << ++a;

Output 7 5 7 4 7 2 7

int a=1, b;

(cout way)

b = a++ + ++a + a + ++a + a++;

5 + 6 = 11

(Ninja way)

cout << a << b

① ③

b = a++ + ++a + a + ++a + a++;

cout << a << b

b = 4 + 3 + 4 + 4 = 15

Be Positive..

a = 5

Scanned with CamScanner

int ch='a';
char ch2='add';
cout<<ch<<"B+"<<ch2

//97 9

jump statements

1) Break
2) continue

for(a=1; a<=10; a++)
{
if(a==5)
break;
else
cout<<a;
}

if(a==5)
continue;

Ship
1 2 3 4 6 7 8 9 10
output

Q-1 Enter values till 0 is entered and display the count of even values, odd, prime values!

int a;
cout<<"Enter value of a = "<<endl;

-1 Nesting of for loop
ii) Print their ASCII value

→ char ch='A';
for(i=1; i<=26; i++)
for(c=1; c<=i; c++)
{
cout<<ch;
ch++;
}
cout<<"\n";

output
A
B B
C C C

OR
char c=1
char c=1

Z Z ... Z

Dt. 7 Pg. [illegible]

```
for(int r=1; r<=5; r++)
{
  for (int sp=4; sp>=r; sp--)
  {
    cout<<" ";
  }
  for (int c=1; c<=r; c++)
  {
    cout<<"*";
  }
  cout<<endl;
}

for(r=1; r<=6; r++)
{
  for(sp=5; sp>=r; sp++)
  {
    cout<<" ";
  }
  for(c=1; c<=(2r-1); c++)
  {
    cout<<"*";
  }
  cout<<endl;
}
```

Dt.
Pg.

```
for(r=1; r<=6; r++)
{
for(sp=5; sp>=r; sp--)
  cout<<" ";
for(c=1; c<=(2*r-1); c++)
{
  if(c==1 || c==(2*r-1) || r==6)
    cout<<"*";
  else
    cout<<" ";
}
```

Ans on pg 21

Q → Display

```
*
* *
*   *
*     *
* * * * *
```

Q) Display

```
+---+---+---+---+
| 1 | 2 | 3 |
+---+---+---+---+
| 4 | 5 | 6 |
+---+---+---+---+
| 7 | 8 | 9 |
+---+---+---+---+
```

Q) Enter a range and display all the Palindrome numbers b/w the range

```
int r1, r2;
cout<<"Enter range";
cin>>r1>>r2;
int num, b;
for(int j=r1; j<=r2; j++)
{
  int rev=0;
  num=j;
  while(num>0)
{
```

Scanned with CamScanner

```
b = num % 10;
rev = rev * 10 + b;
num = num / 10;
}
if (n == rev)
    cout << n << " ";
}
```

Functions: Block of statements performing some task. We can use these statements whenever and wherever req.

Parts →

(1) Declaration → Number of parameter & data type also memory allocated.

Syntax:

```
return_type func_name(parameter/arg list);
```

(2) Definition → task

Syntax:

```
return-type func-name (par list).
{
    ==task;
}
```

(3) Call: `func_name(values);`

(1) non return type and without parameters

main call fn

Eg 01(i)

```
void fun1();  → declaration
void main()
{
  fun1();
  cout << " Inside function" << endl;
fun1();
}
void fun1()
{
  cout << " inside function" << endl;
}
```

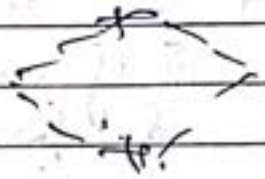
```
void fun1()
{
  cout << "Hello1" << endl;
}
void fun2()
{
  fun1();
  cout << "Hello2" << end;
  void fun3()
{
  cout << "hello3" << endl;
  fun2();
}
int main()
{
  fun3();
}
```

Hello 3
Hello 1
Hello 2

Scanned with CamScanner

Dt. _____ B+
Pg. 24

```
→ for(r=1; r<=7; r++)
{
  for(sp=4; sp>=r; sp--)
  if(r>=1 && r<=4)
{ for(sp=4; sp>=r; sp--)
    cout << " ";
}
  if(r>=5 && r<=7)
{ for(sp=1; sp<=r-4; sp++)
    cout << " ";
}
  for(c=1; c<=2r-1; c++)
{ if(r==1 || r<=4)
  if(c==1 || c==(2r-1))
  { cout << "*";
    else
      cout << " ";
  } if(r>=5 && r<=7)
cout << endl;
}}
```



```
→ for(r=1; r<=7; r++)
{
  if(r>=1 && r<=4)
  { for(sp=4; sp>=r; sp--)
      cout << " ";
  }
  if(r>=5 && r<=7)
  { for(sp=1; sp<=r-4; sp++)
      cout << " ";
  }
}
```

Be Positive...

Scanned with CamScanner

```
for(c=1 ;c<=2*r-1; c++)
{ if (r>=1 && r<=4)
{ if (c==1 || c==2*r-1)
    cout<<"*";
  else
    cout<<" ";
}
if (r>=5 && r<=7)
{ if((c==1 || c==2*r-4)
for (r=7; r<=5; r--)
{ if (c==1 || c==2*r-4)
    cout<<"*";
  else
    cout<<" ";
}
}
cout<<endl;
}

for(r=1; r<=7; r++)
{ if (r%2==1)
{ for(i=1; i<=7; i++)
{ if (i%2==1)
    cout<<"*";
  else
    cout<<"_";
  cout<<endl<<endl;
}
}
if (r%2==0)
{
```

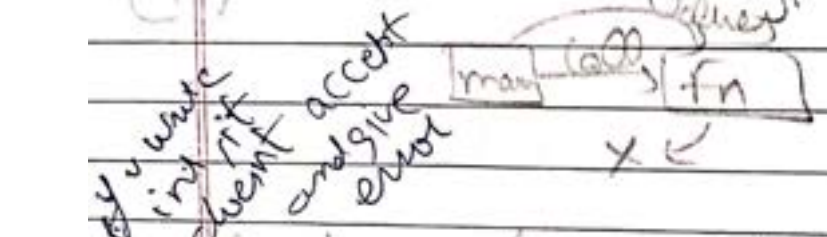
Be Posi

Scanned with CamScanner

Dt.
Pg. 23 B+

```
for (i=2; i<=6; i++)
{   if (i%2 == 0)
        cout<<"1";
    else
        cout<<j<<i
        j++;
    cout<<endl<<endl;
}
```

(3) non return type with parameters



Create a fn accepting two values a displaying them sum.

```
void add (int, int); // dec
int main()
{   int a, b;
    cout<<"enter two values";
    cin>>a>>b;
    add (a, b); // actual arguments
                // call
}
void add (int a, int c)
{   int c;
    c = a+b;
    cout<<"sum = "<<c;
}
```

before main declare if but if u must define

Q- Enter a value and display either its positive number between 1 to 5 if (1-5) (6-10) (>10)

(Number) (6-10) (Table)

2 2
3 3 3
4 4 4 4

and Table of >10

(i) A fn can never be defined inside another fn
2. A variable declared inside a fn has limited scope to that fn only
3. A fn can return a single value at a time.
4. Declaration is e.g. where we define a fn often main

(iii) return type with parameters

main → call → function

Dt.
Pg. 25 B+

Q-> Create a fn accepting 2 values and returning their sum

-> int sum(int, int)
void main()
{
int a, b;
cout << "Enter 2 values";
cin >> a >> b; // call
int d = sum(a, b);
cout << "sum = " << d << endl;
}
int sum (int a, int b)
{
int c;
c = a+b;
return c; // return (c);
}

cout << "output =" << sum(4,8);

Sum of series:

$$s = x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \cdots + \frac{x^n}{n!}$$

cin >> x >> n;
for (i=1; i<=n; i++)
{ s = s + pow(x, i) / fact(i);
}

file.cpp

int fact (int a)
{
int p = 1;
for (int i=1; i<=a; i++)
{
p = p * i;
}
return (p);

Be Positive...

Scanned with CamScanner

file1.cpp → use iostream and namespace std

#include <iostream>
#include "file1.cpp"
using namespace std;
int main()
{
cout << pi;
}

Default argument functions

void abc (int a, int b=10)
{
cout << "sum: " << (a+b);
}

abc(5); //15
abc(5,20); //25

int main()
{
int a=10;
cout << a; //10
cout << &a; //6008
cout << *(&a); //10
}

& → address
* → value

if local variable not found then it will search in same folder for full location

Scanned with CamScanner

Pg. 23 B+

Pointer: A pointer is a variable which points to memory location of another variable

```
int main()
{ int a=10;
  int *p;                    *p=&a; X
  p=&a;                      int *p=&a; ✓
  cout<<p;   //1610.8
  cout<<*p;  //10
  a=20;
  cout<<a<<*p;   //20 20
  *p=30;
  cout<<a<<*p;   //30 30
```

swap by pointer

```
#include
void swap(int*, int*)
int main()
{ int a=10, b=20;
  swap(&a,&b);
  cout<<a<<b;
}
Void swap(int *p1, int *p2)
{ int c;
  c=*p1;
  *p1=*p2;
  *p2=c;
}
```

Scanned with CamScanner

Recursion → function calling itself

Factorial using recursion

```
#include<iostream>
using ____
int fact(int);
int main()
{ int a,b;
  cout<<"enter value"<<endl;
  cin>>a;
  b=fact(a);
  cout<<b<<endl;
}
int fact(int a)
{ if(a==0)
    return 1;
  else return(a*fact(a-1));
}
```

Fibonacci series using recursion

```
void main()
{ int a=0, b=1;
  cout<<a<<" "<<b;
  fab(a,b);
}
  void fab(int a, int b)
{ int c;
```

Scanned with CamScanner

Dt.
Pg. 29 B+

```
void fab(int, int);
int main()
{
int a=0, b=1;
cout<<a<<" "<<b;
fab(a,b);
}
void fab(int a, int b)
{ int c;
  while(1)
{ if(i==10)
  break;
  i++;
  c=a+b;
cout<<" "<<c;
fab(b,c);
}
}
```

Function overloading

```
void abc() ---1
{ cout<<"Hello";
}
void abc(int a) ---2
{
cout<<a;
}
void abc(int a, int b) ---3
{ cout<<(a+b);
}
void abc(float a, float b) ---4
{
cout<<(a+b);
}
int main()
{ abc(10); // 2
  abc(); // 1
  abc(10,20); // 3
  abc(10.5,20); // 4
```

Be Positive...

Scanned with CamScanner

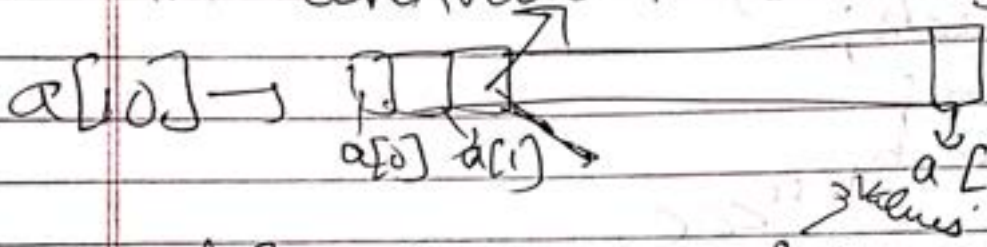
4th type of function return type non-argument

Dt. Pg. B+

→ Array →
Collection of homogeneous data types
int a[30]; → homogeneous

→ Declaration;
int a[10];

→ Elements of an array are stored in continuous memory location



a[0], a[1], ---- a[9] → elements
0, 1, 2, ---- → Index, subscripts

Imp (1) int a[] ✗
int n=10
int a[n] ✗

→ Initialization
int a[5];
a[0] = ; a[1] = ; a[5] = ;

int a[5] = {10, 20, 30, 40, 50}; ✓
int a[] = {10, 20, 30}; ✓
int a[] = {10, 20, 30}; ✓
int a[2] = {10, 20, 30, 40}; ✗

Scanned with CamScanner

Dt.
Pg. 31 B+

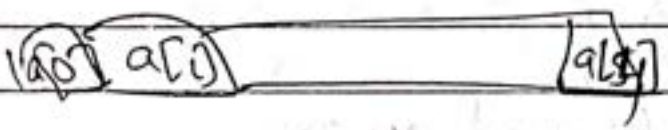
int a[] = {13, 16, 17, 8, 12};
cout << a[2] << endl;

for (int i=0; i<=4; i++)
    cout << a[i] << " "; To display all values

Enter six values in an array and display it

int main()
{
int a[6];
cout << "Enter 6 values";
for (int i=0; i<=5; i++)
    cin >> a[i];
    cout << "values are: ";
for (int i=0; i<=5; i++)
    cout << a[i] << " ";

Largest of ~~eight~~ five values



l = a[0]
for (i=1; i<=4; i++)
{
if (l < a[i])
l = a[i];
}

Scanned with CamScanner

Dt. _____ B+
Pg. _____

Sorting (Ascending or Descending)

| 6 | 9 | 18 | 12 | 7 | 13 | 15 | 8 |

(1) Bubble
↳ a[10] n=6
Consecutive
Value
Compare

```
for(pass=1; pass<=(n-1); pass++)
{
  for(i=0; i<n-1; i++)
  {
    if(a[i] > a[i+1])
    {
      b = a[i];
      a[i] = a[i+1];
      a[i+1] = b[i];
    }
  }
}
```

Selection sort
↳ pass 1st with all.
2nd with all and so on

```
for(i=0; i<n; i++)
{
  for(j=i+1; j<n; j++)
  {
    if(a[i] > a[j])
    {
      b = a[i];
      a[i] = a[j];
      b = a[j];
    }
```

Be Positive...

Scanned with CamScanner

Dt. Pg. B+

reversing an array

```
for(i=0; j=n-1; i<=(n-1)/2; i++,j--)
{ b = a[i];
  a[i] = a[j];
  a[j] = b;
}
```

→ Armstrong is number whose value is equal to sum of cube of its digits → only 3 digit #

Q → Display second largest from n values

Q → Delete duplicate elements from array

Q → Display the count of each element in an array repeated

Q → Enter a range and display armstrong numbers in that range

```
{
  int a, b;
  cout << "Enter range of "a and b (both 3 digit) " << endl;
  cin >> a >> b;

  for(int i=a; i<=b; i++)
  {
    while(i>=0)
    {
      i = i/10;
      c = i%10;
      i = i/10;
```

Q → Create an array containing first n prime values.

Be Positive...

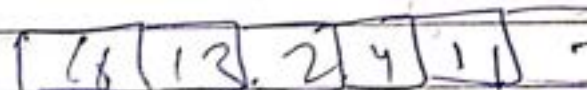
Scanned with CamScanner

Enter n value and display sum

```
int a[100] n;
cout<<"Enter a value:";
cin>>n;
cout<<"Enter "<<n<<" values:";
for(int i=0; i<n; i++)
    cin>>a[i];
int s=0
for(int i=0; i<n; i++)
    s=s+a[i];
cout<<"sum="<<s;
```

Q→ Sorting: ascending or descending

① Bubble sort:



13 16 2 4 11 7 30

2 13 13 13 16 30

```
for(pass=1; pass<=n-1; pass++)
for(int i=0; i<(n-1); i++)
{ if(a[i]>a[i+1])
  {
    b=a[i];
    a[i]=a[i+1];
    a[i+1]=b;
  }
}
```

Scanned with CamScanner

Dt. Pg. 35 B+

Dynamic memory Allocation:

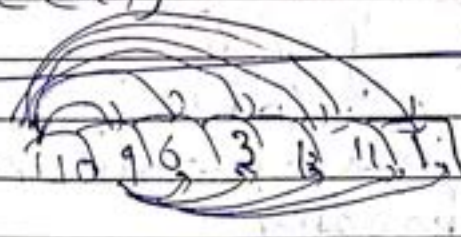
=>

new and delete operator

```
int main()
{ int *a, n;
  cout << "Enter size" << endl;
  cin >> n;
  a = new int[n];
  cout << "Enter n values" << endl;
  for(int i=0; i<n; i++)
    cin >> a[i];
  cout << "Values are:";
  for(int i=0; i<n; i++)
    cout << a[i] << " ";

a = new int[8]; cout << "Again Enter 8 values";
  for(int i=0; i<8; i++)
    cin >> a[i];
cout << "\n Again Values are:";
  for(int i=0; i<8; i++)
    cout << a[i] << " ";
  delete(a);
}
```

(1) Selection sort




```
for(i=0; i<=n-1; i++)
{ for(j=i+1; j<n; j++)
  { if(a[i] > a[j])
    { b = a[i];
      a[i] = a[j];
      a[j] = b;
    }
  }
}
```

Be Positive...

Scanned with CamScanner

for second largest do sorting then pickup the element

Pg. B+

Reverse

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |

```
for(i=0, j=n-1; i<=(n-1)/2; i++, j--)
  { b= a[i];
    a[i]= a[j];
    a[j]= b;
  }
```

~~Deleting duplicate elements~~

```
for (int i=0; i<=n; i++)
{
  for (int j=(i+1); j<n; j++)
  {
    if (a[i] > a[j])
    {
      int b= a[i];
      a[i]= a[j];
      a[j]= b;
    }
  }
}
int i;
for (i=(n-1); i>0; i--)
{ if (a[i] != a[i-1])
  { cout << endl << "Second High" << a[i-1];
    break;
  }
}
if (i==0)
cout << "Second highest not found";
```

Be Positive...

Scanned with CamScanner

int a[][3] = {1, 2, 3}, {4, 5, 6}, {7, 8, 9};
int a[3][] = {1, 2, 3 — Dt. 9} ✗ B+
int a[][] = {1, 2, 3 — Pg. 9}; ✗

Deleting duplicate

```
for (int i=0; i<=n; i++)
{for (int j=(i+1); j<n; j++)
  {
    if (a[i] == a[j])
    {
      for (int k=j; k<(n-1); k++)
        a[k] = a[k+1];
      n--;
      j--;
    }
  }
}
cout << "Distinct values are: ";
for (int i=0; i<n; i++)
  {
    cout << a[i] << "  ";
  }
}
```

int a[5];
output 20000

int a[5] = {2};
a[0] = 2;
output
2 4 Garbage Value

0 → count duplicate elements
0 → store first n prime values
0 → input values 1 to 31 then 1 to 12

(1 to 31) → 4
(1 to 12) → 6

2 3 5 7 11 13 17 19 23 ...
4 6
7 + 13 = 20

Scanned with CamScanner

Dt.
Pg. B+

Arrays and pointers

```
int a[] = {10, 20, 30, 40, 50, 60};
int *p;
p = &a[2];
p--;
for (int i=1; i<=5; i++)
{
    if (i%2==0)
        *p = *p+1;
    else
        *p = *p-2;
    p++;
}
p--;
for (int i=1; i<=6; i++)
{
    cout<<(*p)
    p--;
}
```

28
30
29
31
30
58
51
38
43
18
10

Arrays and functions

Passing individual values

```
int main()
{
    int a[] = {10, 20, 30, 40};
    for (int i=0; i<=3; i++)
        show(a[i]);
}
void show (int x)
{
    cout<<x<<"\n";
}
```

Be Positive...

Scanned with CamScanner

Dt. _____ B+
Pg. 34.

Passing entire array

```
int main()
{ int a[] = {10, 20, 30, 40};
  show(a);
}
void show(int a[])  → Not put 4 here.
{
  for(int i=0; i<=3; i++)
    cout << a[i] << " ";
}
```

or

```
void show(int *p)
{ for(int i=1; i<=4; i++)
  { cout << *p
    p++; }
}
```

or (*p)++ not *p++

```
for(i=0; i<=3(4); i++)
  cout << p[i]
}
```

2-d array

Declaration: int a[rows][columns]

int a[3][4]; declaration

Provide both values in declaration

eg a[3][4] ✓

a[ ][4] ✗

	0	1	2	3
0	a[0][0]			
1				
2				

Initialization

int a[3][3] = {1, 2, 3, ... 9}

int a[3][3] = {{1,2,3}, {4,5,6}, {7,8,9}}

int a[ ][3] = {1, 2, 3 ... 9}; ✓

int a[3][ ] = {1, 2, 3 ... 9} ✗

[ ][ ] = ✗

Be Positive...

Scanned with CamScanner

Dt. _____
Pg. _____ B+

Q → Enter a matrix and display it

```
int main()
{
  int a[3][3];
  cout << "Enter 9 values: ";
  for(r=0, r<=2, r++)
  {
    for(c=0; c<=2; c++)
      cin >> a[r][c];
  }
  cout << "Matrix is: ";
  for(int r=0; r<=2; r++)
  {
    for(int c=0; c<=2; c++)
      cout << a[r][c] << " ";
    cout << "\n";
  }
}
```

→ Input 2 matrices and display their sum

```
int main()
{
  int r, c;
  cout << "Enter no. of rows & columns";
  cin >> r >> c;
  *a = new int[r, ];
```

Scanned with CamScanner

Q- Sum 2 matrices $C_{ik} = a_{ij} b_{jk}$

Q- Multiply 2 —

Dt. Pg. B+



2×4 4×2 2×2

Dynamic matrix

```
int main()
{
int **a, **b, c, j
cout<<"Enter no of rows and columns: ";
cin>>a[i][j]>>c[i][j]
cout<<c[i];
a = new int *[r];
for (int i=0; i<r; i++)
a[i] = new int [c];
cout<<"Enter"<<(r*c)<<"values: ";
for (int i=0; i<r; i++)
{
for (int j=0; j<c; j++)
cin>>a[i][j];
}
cout<<"Matrix is: \n";
for (int i=0; i<r; i++)
{
for (int j=0; j<c; j++)
{
cout<<a[i][j]<<" ";
}
cout<<endl;
}
int a[3][3] = {1, 2, 3, 4, 5, 6, 7, 8, 9};
cout<<a; // Base add of a
cout<<&a[0]; // Base add of
cout<<a[1]; // Base add of
cout<<&a[1][2] // 6
```

Be Positive..

Scanned with CamScanner

Dt.
Pg.

int a[][3] = {1, 2 ——— , 9};
show (a);

void show (int *p) — or int **p
{ p[i][j]
for (int i=1; i<=9; i++) ↓ in loop.
{ if (i==1 || i==4 || i==7)
cout << endl;
cout << *p; — or *(p++);
p++;
}
}

or pass to 2d array a[][3] ↓ yes no[][] no[3][]

Note: 1) In ... by pointer function ... need to write ...

2 4 2 2

a=121 b=240

i	sum	c	k
121	0	121	1
122	1	12	2
	3	1	
	10	0	

121
122

122

2 4 3 2 3

R 1 2 3 4 43 B+

Dt. Pg.

Store R prime values in array

```
#include<iostream>
using namespace std;
int main()
{
    int n, *p, ind=0;
    cout<<"Enter n:";
    cin>>n;
    int val=1;
    p=new int[n];
    do
    {
        int count=0;
        for(int i=1; i<=val; i++)
        {
            if(val%i==0)
                count++;
        }
        if(count==2)
            p[ind++]=val;
        val++;
    }while(ind<n);
    cout<<"Prime values:";
    for(int i=0; i<ind; i++)
        cout<<p[i]<<" ";
}
```

Scanned with CamScanner

Dt.
Pg.

'a' "a" "a"

Variable character string

→ String - set of characters or array of characters

char a[10] - it can store 10 characters including NULL('\0')

0 1 2 3 4 5 6 7 8 9

Declaration of a[10]; not a[]

Initialization

char a[10] = "Hello";

char a[] = "Hello";

char a[2] = "Welcome"; ✗

→ char a[4] = abcd is not allowed in C (not in Java)

Diff of 1 or 2 is allowed

char a[] = "Welcome"

char a → base address (of w)

cout << a; // Welcome

cout << a[0]; // w

Dt.
Pg. 45 B+

a[3] - 4th character (element
a -> base address (add of w)

To display only one character

p = &a[2]
cout << *p;

char a[] = "Hello", b[10];
b = a; ×
(means to store base addr of a in b not working)

char a[10];
a = "Hello"; ×

a[] = "Hello"; × (Array)

char a[10];
a[0] = 'H';
a[1] = 'e';

a[10] = 'l';

Enter a string and display it

int main()
{
char a[10];
cout << "Enter your name:";
cin >> a;

Be Positive..

Scanned with CamScanner

Dt.
Pg.

Jasjot Singh
only output Jasjot not Singh
cout << "Hello" << endl;
}

Inbuilt functions:
#include <string.h>

(1) strlen() → provide length of string

char a[]: "Hello";
int l;
l = strlen(a);
cout << l; // 5

(2) strrev()
reverse string characters

char a[]: "Hello";
strrev(a);
cout << a; // olleH

(3) strupr()
uppercase

char a[] = "hello";
strupr(a);
cout << a; // HELLO

(4) strrev()
strrev(a);
cout << a; // olleH
reverse

B+

(5) strcpy()

#strcpy (target, source

char a[10] = "Hi"; b[10] = "welcome";

strcpy(a, b);

cout << a << b; // welcomewelcome

#strcat

#strcat (target, source)

char a[10] = "Hello";

char b[] = "welcome";

strcat(a, b);

cout << a << b; // Hellowelcome

#strcmp: → compare alphabetically

char a[] = "Push";

char b[] = "Pull";

int x;

x = strcmp(a, b);

if (x > 0)

cout << "a";

(Ethiopia
Hello is
not
Hello)

Be Positive...

Scanned with CamScanner

Default ⇐ Unsigned 2⁰ – 2⁷ i.e 255

Signed → 127-0+128

Dt. B+

```
if(x<0)
  cout<<"2nd";
if(x==-0)
  cout<<"Equal";
```

if at+ ⇒ error.

```
int main()
{
char a[]="welcome";
show(a);
}
void show(char *s)
{ cout<<s; // welcome
  *s // w
```

welcome\0

s

to display welcome show(a[3])

or

```
void show(char *s)
{
for(int i=0; i<strlen(s); i++)
  cout<<s[i];
}
```

s[i]!='\0'

Not do show(a) wrong

or

```
void show(char *s)
{
while(*s!='\0')
  cout<<*s;
  s++;
}
}
```

Scanned with CamScanner

Dt.
Pg. 49 B+

1) strlen

```
int main()
{ char a[]="Welcome";
  int b;
  b=stringlen(a); // call
int stringlen(char *s)
  { int c=0;
    while(*s!=0)
    { c++;
      s++;
    }
    return(c);
  }
```

```
int main()
{
 char a[]="Welcome";
  //strupper(a);
   cout<<a;
}
void strupper(char *s)
  {
   while(*s!='\0')
   { if(*s>='a' && *s<='z')
       *s=*s-32;
     s++;
  }}
```

Be Positive...

Scanned with CamScanner

Dt.
Pg. S.O. Bt

```
int main()
{
char a[]="welcome";
strrev(a);
}
void strrev(char *);
{
for(int i=0; j=(strlen(s)-1)
    j<=(strlen(s)-1)/2; i++,j--)
{ a=s[i];
  s[i]=s[j];
  s[j]=a;
}
}
int main()
{ char a[10]="Hello";
  b[]="welcome";
strcpy(a,b);
}
void strcpy(char *,char *)
{
while(*s2!='\0')
{ *s1=*s2; s1*(s1++)
  s1++; =*(s2++)
  s2++;
*s1='\0';
```

Scanned with CamScanner

S.No. Date Title Page No. Remarks

```
int main()
{char a[]="hello"
show(a);
}
void show(char *s)
{
cout<<(*s)++;      // h
cout<<s;           // iello
cout<<*(s+e);      // h
cout<<s;           // ello
cout<<*(s)++;      // h
cout<<s;           // ello
```

Calenation

```
void strcat(char *s1,char *s2)
{ while(*s1!='\0')
  s1++;
while(*s2!='\0')
{*s1=*s2;
 s1++;
 s2++;
}
*s1='\0';
}
```

Scanned with CamScanner

strcmp()

Dt. Pg. 53 B+

```
char a[] = "Ruth", b[] = "Rull";
int x;
x = strcmp(a, b);
if (x<0) 2nd word < 1st word
if (x==0) equal
int strcmp(char *s1, char *s2)
{ while (*s1 != '\0' || *s2 != '\0')
  { if (*s1 != *s2)
      return(*s1 - *s2);
    s1++;
    s2++;
  }
  return(*s1 - *s2);
}
```

→ hello is not equal to Hello but make a prog to make them equal or any other result strcmp()

→ pass = ABCD if abcD
Key = AMAN] output ABCD
encKey = BOBR

Be Positive...

Scanned with CamScanner

Dt.
Pg. 54 B+

interchange
if ex: if > Z then -26 in your key.
else put '\0' at end.

int main()
{ char a[20];
cout << "Enter your name";
cin >> a;
cout << "Hello" << a;
}

→ set string.
gets()
puts() → display string

→ cout << "Enter your name:";
gets(a); → same as cin
cout << "Hello" << a; or puts(a);

2-d string
char [3][10]; → 3 strings having max 9 characters

0→9
A→F
10→19
1A→1F
20→29
2A→2F
etc

Initialization
char a[3][10] = {"Hello", "Bye", "welcome"};
cout << a[2][1]; // e
cout << a[2]; // welcome
cout << a; // address

```
char a[3][10];
cout<<"Enter 3 names:";
for(int i=0;i<=2;i++)
  cin>>a[i]; or gets(a[i]);
```

Functions → show() → void, show(char a[][10]) and all methods of 2d array pointers and double pointers

Q→ Input 5 n strings and display alphabetically (by 2d strings) dictionary

Q→ Input 5 strings of 10 characters and create password of 10 digits by choosing 2 from each 5 but no duplicate
eg (a b) c ——
ab (3 3 4)

Oops concepts
object oriented programming system
1) Modularity → not functions (or it means classes, constructors etc)
2) Reusability → Inheritance and Polymorphism
3) Security → public, private, protected data

Scope resolution (::) →

```
int a=10;
int main
{
  int a=20;
  cout<<a; //20
  cout<<::a; //10
  a=30;
  ::a=90; cout<<a<<::a //3090
```

Be Positive...

Scanned with CamScanner

Dt. _____
Pg. _____ B+

class es RollNo Name Marks
int a;
int a[30];
User defined data types which can deal with heterogenous types of data, class is basically a template
eg 1)
Class contain 2 members
↓ Data Members (Variables)
↓ Member functions (Functions)

object → example, Entity or Instance of a class (same meaning)
memory to data member of class will be allocated if we create its object(s). Objects will access the members of class using dot operator.

Class
Fruit
Color
Taste
Price

Object
↓
Mango
Apple

eg 2)

Dt.
Pg. 57 B+

Defining a class

```
class cls_name
{
  data members,
  member functions
};
```

Object → example
datatype → class

cls_name obj_nm;

eg 1 →

```
class Abc
{ public:
  int a, b; // Data Members
  void show()   // Member function
  {
    cout<<a<<b;
  }
};
  int main()
{
  Abc ob1;
  Abc ob2;
  ob1.a = 10;
  ob1.b = 20 → input at call
  ob2.a = 30;
  ob2.b = 40;
  ob1.show(); // 10 20
  ob2.show(); // 30 40
```

eg 2 →

```
class Abc
{ public:
  int a, b;
    void get()
    { a = 10;
      b = 20;
    }
```

Be Positive...

Scanned with CamScanner

```
void show
{
  cout<<a<<b;
}
};
int main()
{ Abc ob1,ob2;
  ob1.get();
 ob2.get();
  ob1.show();
  ob2.show();
}
```

```
class ABC
{
int r_no;
char section;
float marks;
public:
 void get()
  { cin>>r_no>>section>>marks;
};
void show()
{
cout<<r_no<<section<<marks;
};
int main()
{ Abc *t1, t2;
  t1.get();
  t2.get();
  t1.show();
  t2.show();
```

not allowed

Scanned with CamScanner

Dt.
Pg. 51 B+

eg 3

```
class Abc
{ char nm[20];
  int marks;
  public:
void get (int a, char *b)
  {
    marks = a;
    strcpy (nm, b);
  }
void show()
{ cout << nm << "\t" << marks << endl;
}
};
int main()
{ Abc a1, a2;
  a1.get (87, "Abcd");
  a2.get (81, "XYZ");
  a1.show();
  a2.show();
}
```

declaring out side not inline

```
void Abc :: get (int a,
{ marks = a; char *b
  strcpy (nm, b);
}
```

inline means it comes to main and no wastage of time in moving from one memory to others whereas

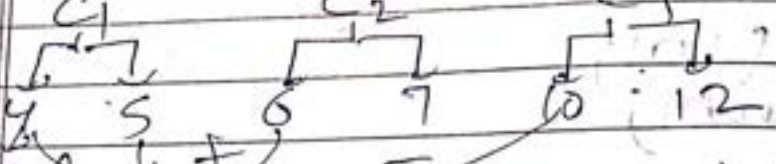
inside is defined inline & outside not inline

Be Positive...

Scanned with CamScanner

Dt.
Pg.

Objects as parameters:




```
class comp
{
  int rl, im;
  public:
  void get(int a, int b)
  {
    rl = a;
    im = b;
  }
  void show()
  {
    cout << rl << "+i" << im << endl;
  }
};
int main()
{
  comp c1, c2, c3;
  c1.get(4, 5);
  c2.get(6, 7);
  c3.add(c1, c2);
  c3.show();
}
void add(comp s1, comp s2)
{
  rl = s1.rl + s2.rl;
  im = s1.im + s2.im;
}
```

Scanned with CamScanner

Dt.
Pg. 61

Array of object

```
class Abc
{ public
  char nm[20];
  int marks;
  void get()
  { cin>>nm>>marks;
  }
  void show()
  {
  cout<<nm<<endl<<marks<<endl;
  }
};
int main()
{ Abc s[6]
  cout<<"Enter names and marks of 6 students";
  for(int i=0; i<=5; i++)
    s[i].get();
  cout<<"Details are:";
  for(int i=0; i<=5; i++)
    s[i].show();
}
```

Enter name and marks of n students and display highest marks, display name of it with highest marks and average of the class

~~Abc obj~~
~~obj = s[0]~~
~~for(int i = 1~~

Abc ob
ob = s[0]; initialise
~~Abc~~ s;
s = new abc[n];

We can make objects of class elsewhere except main

Friend function

A normal function outside the class which can access private members of class

Scanned with CamScanner

friend complex add (complex &, complex &);

Same using friend fn

complex add (complex &, complex &)
{ complex t;
t.a = a1.a + a2.a;
t.ib = a1.ib + a2.ib;
return t; }

int main()
{ c3 = add(c1, c2) }

Eg class ABC
{ int a, b;
actually 50,000 taken

ACC. No → bank → if duplicate ACC no is given 10,000 deposit

(U1) U2 U3

10K 10K 10K Same

same → deposits 10K in ACC

Static data Member => These are allocated single memory for various users. The are automatically initialized

eg class Abc
{ static int a;
public:
void show()
{
i++;
cout << a;
}
};
int Abc :: a;
int main()
{ Abc i, j, k;

Be Pos

Scanned with CamScanner

Dt.
Pg. 63 B+

ob1.show(); //1
ob2.show(); //2
ob3.show(); //3

→ Constructors are special member functions which are used to construct data member of class. They are special : their name is same as class name. They do not have any return type : They are automatically invoked when we create object of class. Always public

```
class Abc
{ public:
    Abc()
    {
      cout<<"Hello"
    }
};
```

invoke

```
int main()
{ Abc ob1;        //Hello
  cout<<"Main"; //Main
  Abc ob2, ob3; //Hello Hello
```

Default

```
class abc
{ int a,b;
  public:
  abc()
  { a=10, b=20;
  }
  void show()
  { cout<<a<<" and "<<b<<endl;
  }};
```

```
int main()
{ abc ob1;
  abc ob2, ob3;
  ob1.show();
  ob2.show();
  ob3.show();
}
```

Scanned with CamScanner

```
class ABC
int a,b;
public:
abc()
{
cout<<"Enter two values";
cin>>a>>b;
}
void show()
{ cout<<a<<"and"<<b<<endl;
}
int main()
abc ob1;
abc ob2, ob3;
ob1.show();
ob2.sho
ob3
}
```

Parameterised constructor

```
class abc
{ int a,b;
public:
abc(int x, int y)
{ a=x;
b=y;
}
void show()
{ cout<<a" and "<<b<<endl;
}
int main()
{ abc ob1(10,20);
abc ob2(2,67);
ob1.show();
ob2.show();
}
```

Scanned with CamScanner

Copy Constructor.

Dt. 05 B+
Pg.

```
class abc
{ int a;
  char b[20];
public:
  abc(int x, char *y)
  { a=x;
    strcpy(b,y);
  }
void show()
{ cout<<a<<" and "<<b<<endl;
}
abc(abc &S1)        // to make many time
{ a=S1.a;
  strcpy(b, S1.b);
}
};
int main()
{
  abc ob1(10, "Hello");
  abc ob2(ob1);     // copy constructor  // implicit call
  ob1.show();       // abc ob2=ob1;
  ob2.show();       // explicit call
}
```

overloading

```
Class abc
{ int a, im;
public:
  void show()
  { cout<<a<<"+i"<<im;
  }
  abc(int a, int b)
  { a=a; im=b;
  }
};
```

abc(abc &, abc)
{ Bc positive abc.im+abc.

```
abc (abc &s1, abc &s2)
{ rl = s1.rl + s2.rl
  im = s1.im + s2.im
};
int main()
{ abc ob1 (10, 20);
  abc ob2 (2, 7)
  abc ob3 (ob1, ob2);
    ob1.show();
    ob2.show();
    ob3.show();
}
```

constructor + friend fn.

```
class abc
{ int rl, im;
  public:
    void show()
    { cout << rl << "+i" << im << endl;
    }
friend abc add (abc, abc);
  abc(int a, int b)
{ rl = a;
  im = b;
}; abc() { } };
  abc add (abc &s1, abc &s2)
{ abc s3;
  s3.rl = s1.rl + s2.rl
  s3.im = s2.rl + s2.im;
  return (s3);
```

Scanned with CamScanner

#include <bits/stdc++.h>

Pg. 67 B+

```
int main()
{  abc ob1(10,5);
   abc ob2(4,7);
abc ab3;
   ob3 = add(ob1,ob2);
   ob3.show();
}
```

Pertinctors → Destructors are used to destroy the values and memory of data members. Always public

~~Ca~~ ~abc()

(Hide)

Destructors are invoked when the scope of an object is terminated or ove

Only one.

```
Class Abc                    output
{ static int a;              1
public:                      2
  Abc()                      3
  { a++;                     2
    cout<<a;                 3
  }                          4
~Abc()                       3
  { a--;                     2
    cout<<a;                 1
  }                          2
};                           1
                             0
```

```
int Abc::a;
int main()
{ Abc ob1;
  { Abc ob2;
    { Abc ob3; → Sepha of ob
    }
    Abc ob4, ob5;
  }
  abc(ob6);
}
```

Scanned with CamScanner

static / friend / destructor

Ant.tech@hotmail.com

Dt.
Pg.

→ we have to give some condition in destructor (compulsory) and it will not free memory It is just invoked when object goes out of scope.

Inter Operator Overloading

cout << → operator overloading
a << 3
↓ bit shift

objects
c1 ← → c2
4 6 8 3

$C_3 = C_1 + C_2$

Return type operator +|-|>= (parameters.)
↓ make it member or friend fn
not in main ∴ it has to access private member

→ Unary operator :

Member fn : No parameter
Friend fn : Single —

Dt. Pg. 69 B+

Binary
Memb. func : Single parameter
friend. — : Two —

object (ob)
↓10 -20 ↓30

-obj
-10 20 -30

Member fn
Unary

```
class Abc
{ int a,b,c;
  public:
    void get(int x, int y, int z)
    { a=x; b=y; c=z; }
    void show()
    {
      cout << a << b << c;
    } void operator -() { a=-a; b=-b; c=-c; }
int main()
{ Abc ob;
  ob.get(10,-20,30);
    -obj
  ob.show(); // -10, 20, -30.
}
```

friend fn Unary friend void operator -(Abc &); inside class

```
void operator -(Abc &s1)
{ s1.a = -s1.a;
  s1.b = -s1.b;
  s1.c = -s1.c;
}
```

-ob;

Abc ob;

Abc c

Be Positive...

Scanned with CamScanner

Dt.
Pg.

Binary number

```
class complex
{ int rl, im;
  public:
  void get(int a, int b)
  { rl = a;
    im = b;
  }
  void show()
  { cout << rl << "+i" << im << endl;
  complex operator+ (comp s)
  { comp M;
    M.rl = rl + s.rl;
    M.im = im + s.im;
    return M;
  }
int main()
{
  complex c1, c2, c3;
  c1.get(4,6);
  c2.get(5,6);
  c3 = c1 + c2; → parameter
     ↳ Invoke
  c3.show();
}
```

Binary friend (yourself)

```
class Abc
{ char *s;
  public:
  void get(char *t)
  {
    s = new char[strlen(t)+1];
    strcpy(s,t);
  }
  int operator>=(Abc x)
  { if (strlen(s) >= strlen(x.s))
      return(1);
    else return(0);
  }
```

Be Positive...

Scanned with CamScanner

<string> some more stuff

<string>
<string.h>

Dt.
Pg. 71

// we can also compare strings using s1.strcmp

int main()
{ Abc a1, a2;
s1.get("Hello");
s2.get("welcome");
if(a1>=a2)
cout<<"1st";
else
cout<<"2nd";
}

istream
ostream (cout)
cout(obj);

overloading insertion and extraction operator never mention in exam later

Note → Always using friend function

class Abc
{ int a,b;
public:
friend istream& ope ___;
friend ostr ___;
};

int main()
{ Abc ob1;
cin>>ob1;
cout<<ob1;
}

istream& operator>>(istream& in, Abc &ob2)
{ ob1>>ob2.a;
ob1>>ob2.b;
return(ob1);
}

necessary as istream is used

ostream& operator<<(ostream& out, Abc &ob2)
{ a1<<ob2.a;
and
a1<<a2.b;
}

Be Positive!

Dt.
Pg.

Not member fn :: if we use member fn then it will call object only, so ....

#include <conio.h> → getch()
getche()
clrscr()

console : input output screen

1) getch() → It accepts single character without displaying it and will automatically jump to next statement

eg char ch;
cout << "enter a key";
ch = getch();
cout << "You have entered" << ch;
Jumps to next

Usage

#include <iostream>
#include <conio.h>
using namespace std;
int main()
{ char ch;
cout << "enter";
for(int i=1; i<=4; i++)
{ ch = getch();
cout << "*";
}
cout << "done";

Be Positive...

Scanned with CamScanner

4 digit PIN

Dt. 73 B+

```
int main()
{ char ch;
  int pin=0;
  cout<<"Enter pin";
  for(int i=1; i<=4; ++i)
  { ch=getch();
    cout<<"*";
    ch=ch-48;
    pin=pin*10+ch;
  }
  cout<<endl<<pin;
  cout<<" done";
  cout<<"Enter any key to terminate";
  getche();
  cout<<"bye";
}
```

clrscr → in gcc compiler

1) getche() → It will delay also

ff.

system("CLS"); it will clear screen and we have to use library

#include<stdlib.h>

#include<iomanip.h> strictly iomanip

input output manipulators

```
int a=3142;
cout<<a; //3142
cout<<setw(8)<<a; // ---3142
cout<<setw(2)<<a; //3142
```

no fix — Be Positive... — can't count on 8th place

float a = 3.142;
cout << a; // 31.419999
total 4
cout << setprecision(4) << a; // 3.142
cout << fixed << setprecision(2) << a; // 31.42
must to proceed
for fixed 31.419999
only fixed use
full 6 by default
If setprecision(8) and only 3.6f
or if fixed << setprecision(8) then
then 31.64999984
garbage
int a = 3142;
cout << setw(10) << setfill('*') << a;
after ****3142
macros it means we have to perform before main
defined
preprocessor
eg: # define a 3.14
if u change value of a then error

Dt.
Pg. 75 B+

Q (Input 2 date and count # of days blw them)

#define PI 3.14 ; pickup any data type

#define area(r) 2*PI*r → faster as compared to function but drawback is we cannot make lengthy

int main()
{ cout << area(2);
}

→ Study yourself
#ifdef, #ifndef
If defined if not defined
Diff blw macros and functions

Project ATM System

Enter Pin : * * * * → 3 trials
if correct entered
Incorrect

Choose an option :-
1. Balance Enquiry
2. Cash withdrawal
3. Change PIN
4. Exit

Thanks for Banking with us

→ Available Amount : 20000 ↵ → Not both till enter.
→ Enter Amount : 10000.
Available Amount : 10000 ↵
→ Current Pin : * * * * * 3 trials
New Pin : * * * * (Restricted)
Confirm Pin : * * * *
Successfully changed! ↵

Be Positive...

Scanned with CamScanner

→ works on all

Template classes

```
#include<iostream>
template<class T1>

Void abc(T1 a, T1 b)
{ T1 c;
  c=a+b;
  cout<<c;
}
int main()
{ abc(10,20);
  abc(10.5,20);
}

template<class T>
  class Abc
{ Ta,b;
  Public:
  Void get(Tx, Ty)
    { a=x;
      b=y;
    }
  Void show()
  { cout<<a<<b;
  }
  int main()
  {
      Abc<int> ob1;
      Abc<float> ob2;
      ob1.get(10,20);
      ob2.get(30.4,60);
}
```

Scanned with CamScanner

```
template<class T1, class T2>
class Abc
{ T1 a;
  T2 b;
public:
  void get(T1 x, T2 y)
  { a=x; b=y; }
void show()
{ cout<<a<<b; }
int main()
{ Abc<int, float> ob1;
  Abc<int, int> ob2;
  ob1.get(10, 20.5);
  ob2.get(30, 60);
}
```

Exception Handling :-

1. try block → The statements prone to an exception are written inside try block
2. throw statement → It will throw an exception from try block towards catch block.
3. Catch block → The block will handle the thrown exception

```
int main()
{ int a,b,c;
  cout<<"Enter two values: ";
  cin>>a>>b;
  try
  { if(b==0)
      throw(b);
    else
      cout<<"Output="<<(a/b);
  }
```

In other languages only try block in C++ everything

Be Positive...

Scanned with CamScanner

catch immediately after try : if 4-5 try catches then it will confuse

```
catch (int e)
{ cout << "Can not be divided by" << e;
}
```

block necessary if single statement not needed

```
int a;
cin >> a;
try
{ if (a%2 == 0)
    throw (a);
  else if (a%3 == 0)
    throw ("Hello");
  else
    throw (10.5);
}
catch (int b) - catch corresponding to every throw
{ cout << b;
}
catch (char b)
{ cout << b;
}
catch (float b)
{
  cout << b;
}
```

Try catch(T b) using template

global catch :

```
catch(...)
{ cout << "Exception Occured";
}
```

Be Positive...

If we h
Int separ
it catch
If int
If it i

Inheritanc

A → B

class

Public

public

In both

Types :

(4) Hierarc

#inc

Scanned with CamScanner

Dt.
Pg. 79 B+

If we have to call int separately and all other globally then int catch must be above global catch if int catch below global catch then it will not go to int catch

→ Inheritance: Reusability

A → Parent / Super / Base / old class

B → child / Sub / New / Derived class

Syntax: class B : visibility mode A

Private → public of A become private of B

Public → All public members of B class become public of new class

In both cases private members NI

Types:- (1) Single (2) Multilevel (3) Multiple

(4) Hierarchical (5) Hybrid - mixture of Hierar

```
#include
using
class Abc
{
  void show1()
  { cout<<"Private";
  }
  public:
  void show2()
  { cout<<"Public";
  }
};
```

Be Positive..

Scanned with CamScanner

Dt.
Pg.

```
class xyz : public Abc
{ public:
  void display()
    { cout << "derived class";
    } };
int main()
{ xyz obj;
  ob.display();
  ob.show();
}
class A
{
  void show_A1()
  { cout << "Hello 1";
  }
  public:
  void show_A2()
  { cout << "Hello 2";
  }
};
class B
{ void dis1()
  {
    cout << "Hello 3";
  public:
    void dis2()
    { cout << "Hello 4";
    } };  not public A, B
class C : public A, public B
```

Scanned with CamScanner

Constructor in Inheritance

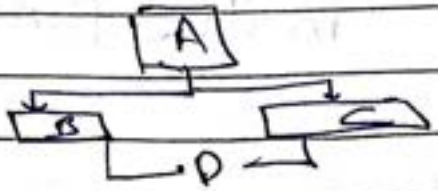
```
class A                          class B : public A
{ int a, b;                      { int c;
public:                            public:
  A(int x, int y)                  B(int x1, int x2, int x3)
  { a=x; b=y; }                      : A(x1, x2)
  void show()                      { c=x3; }
  { cout<<a<<c<<b; }               void show1()
};                                 { cout<<c; }
                                   int main()
                                   { B ob(10,20,30); }
```

Multiple

```
class A                          class C : public A, public B
{ int a, b;                      { int d, e;
};                               public:
class B                            C(int x, int x2, int x3, int x4, int x5)
{ int c;                             : A(x, x2), B(x3)
};                                 { d=x4; e=x5; }
                                   int main()
```

Protected, Protected members, they act as private members for some class and can be inherited

If ob of D Hybrid Inheritance



Scanned with CamScanner

D → show-B
show-A
show-C
show-A

```
class A
{ public:
  void show A()
  { cout<<"A-class";
  }
};
class B: public A
{ public:
  void show_B()
  { cout<<"B-class";
  }
};
class C: public A
{ public:
  void show_C()
  { cout<<"C-class";
  }
};
class D: public B, public C
{ public:
  void show_D()
  { cout<<"D-class";
  }
};
int main()
{ D ob;
  ob.show_D();
  ob.show_B();
  ob.show_C();
  ob.show_A(); → error ambiguous
}
```

public or virtual → get 1st path, neglect other

we have to do virtual to both paths to prevent ambiguity

Polymorphism → one name many forms

Compile Time (Early or Static Binding)
- Function overload
- Operator overload

Run Time (Late or Dynamic Binding)
- Virtual function

Scanned with CamScanner

virtual means false

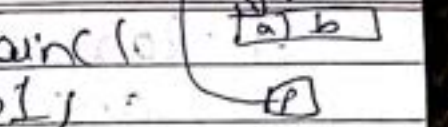
Dt. Pg.

B+

Object as Pointer

```
class Abc
{ public:
  int a, b;
  void show()
  { cout<<a<<b; }
};
```

```
int main()
{ Abc ob1;
  ob1.a = 10; ob1.b = 20;
  Abc *p;
  p = &ob1;
  ob1.show(); // 10 20
  (*p).show(); or p->show(); // 10 20
  p->a = 20;
  ob1.show(); // 20 20
```




```
class Abc
{ public:
  void show()
  { cout<<"Base"; }
};
class Xyz : public Abc
{ public:
  void show()
  { cout<<"Derived"; }
};
int main()
{ Abc ob1;
  Abc *p;
  p = &ob1;
  p->show(); // Base
  Xyz ob2;
  p = &ob2;
  p->show(); // Base
```

if we write virtual void show then Derived.

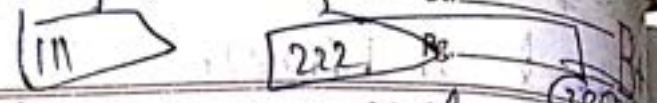
p = (Xyz)

Be Positive...

Scanned with CamScanner

Bank ID Dist.

Virtual class



Abstract class → A class in which some or all the members are only declared, and these declared members are further defined by sub-class.

```
abstract class Abc
{ public:
  abstract void show1();
  void show2().
  { cout<< "Non-Abstract";
  }};
class XYZ: public Abc
{ public:
  void show1()
  { cout<< "Function defined!";
  }
  void display()
  { cout<<"Hello"<<endl;
  }};
```

In same manner some other class can define void show1() whatever publically.

Virtual class: All except pure abstract class is the lowest known as virtual class. We can't create its object ∵ it doesn't define any fn.

No private members

This pointer →

~~void show()~~

Refers to an invoking object

Refer to object which invoked

Dt. 8 → B+
Pg.

```
class Abc
{ int a, b;
  public:
  void get(int a, int b)
  { this->a = a;
    this->b = b;
  }
  void show()
  { cout << a << b; }
};
int main()
{ Abc ob1;
  ob1.get(10, 20);
  ob1.show();
}
```

a of ob
→ data member
otherwise Compiler confuse
cout << this->a << this->b;

```
class Abc
{ char *a;
  public:
  Abc void show()
  { cout << a; }
  int void great(Abc s2)
  { int c, d;
    this->s1
    c = strlen(s2);
    d = strlen(this->s1);
    if (c > d)
      return c;
    else
      return d;
  }
  void get(Abc s3)
  { a = s3; } };
```

```
int main
{ Abc S1("Hello");
  Abc S2("welcome");
  Abc S3;
  S3 = S1.great(S2);
  S3.show(); // welcome
}
Abc(char *s)
{ a = new char[strlen(s)+1];
  strcpy(a, s);
}
Abc great(Abc &S)
{ if (strlen(S.a) > strlen(a))
    return(s);
  else
    return(*this);
} };
```

Be Positive...

Scanned with CamScanner

Similarly calloc but it initialize 0 too allocated

int *p;
p = new int[6];
or p = new int[10];
p = (int *) malloc(6);
p = (int *) realloc(p, 10);

Dt.
Pg.

→ File handling

1) Open file
2) Task perform
3) Close.

string ← fstream ← file stream

fstream (library)

input ← ifstream (reading) — output ← ofstream (writing) (closed) objects

1) get()
2) put()
3) getline()
4) open()
5) close()
6) tellg()
7) seekg()
8) read(), write()

1) get() → It reads single character from a file and jumps to next character
2) put() → writes single character in a file
3) getline() → Reads a string from a file
4) open() → Used to open file.

Syntax: open("filename.ext", mode);

abc ← abc.cpp
abc.txt

optional
ios::in
ios::out
ios::app
ios::ate
ios::binary

append ios::app

or ios::ate

5) Close →
6) Tellg
7) Seekg
8) Then

Be Positive

Scanned with CamScanner

5) Close → To close file

6) Tellg, tellp, provides location of file pointer

7) Seekg / seekp, we can provide the location or position using these

8) These are used to read and write in binary form

```
#include<fstream>
using namespace std;
int main()
{ ofstream ob;
  ob.open("basic.cpp");
  ob<<"Hello\n";
  ob<<"This is file handling";
  ob.close();
}
```

→ overwrites deletes data but no one can open if not closed
i.e write time it creates
if → ofstream it creates file if not already exist
if if → read ex must exist

if we write ("basic.cpp", ios::app); it does not overwrites but retains

default is ios::out

Be Positive...

Scanned with CamScanner

#include <iostream>
#include <fstream>
using
int main()
{
ifstream ob;
ob.open("array 1.cpp"; ios::in);
char ch; while(ob)
{ ch = ob.get();
cout << ch;
}
ob.close()
}

class.cpp → arithmetic.c

read | write

int main()
{ char f1[30], f2[30];
cout << "enter first file name";
cin >> f1;
cout << f1;
cin >> f2;
ifstream ob1;
ob1.open(f1); member fn
ofstream ob2;
ob2.open(f2); constructor overloading
or ofstream ob2(f2); constructor
char a; while(ob1)
{ a = ob1.get();
ob2 << a;
or ob2.put(a);
}

ex.
ob2(f2, ios::app)

Be Positive

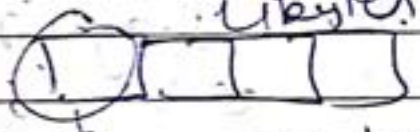
Dt.
Pg. 89 B+

obj.close();
obj2.close();
}

we can use object of fstream also in main and mode is (ios::in | ios::out)

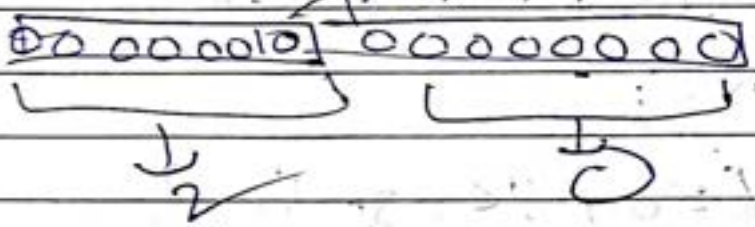
read( )
write( ) → Binary form

float a = 16.34



use all bytes separately

int a = 512;



int a = .23

2 * .23 = 0.46	0
2 * .46 = .92	0
2 * .92 = 1.84	1

16.23
10000.001

Scanned with CamScanner

#include <iostream>
#include <fstream>
using
int main()
{
float a[] = {2.3, 4, 88, 2.3, 56, 20, 14};
fstream obj;
obj.open("filenew.txt", ios::out | ios::binary);
obj.write((char*)&a, sizeof(a));
obj.close();
fstream ob2("file-new.cpp", ios::in | ios::binary);
ob2.read((char*)&a, sizeof(a));
ob2.close();
for(int i=0; i<6; i++)
cout << a[i] << " ";

tellg() getpos
tellp() putpos

fstream ob("abc.c");
while(ob)
{ ch = ob.get();
if (ch == 'c')
cout << ob.tellg();
}

characters printed

numbers
strings
if comments
or with main

Be Positive

Scanned with CamScanner

→ ofstream

tellp() for write

tellg() for read

Dt.

Pg. 9 B+

#include <iostream>
#include <fstream>
using namespace std;
int main()
{
float a[] = {6.5, 8.8, 7.1};
ofstream ob1("file-new.txt", ios::app);
ifstream ob2("file-new.cpp");
char ch;
while(ob2)
{ ch = ob2.get();
if(ch == 'e')
cout << ob1.tellp() << endl;
}
ob1.close();
ob2.close();
}

(writing starts from 0)

seekg() | seekp()

→ ofstream

→ ifstream

seekg() (offset, location)

ios::beg (immediate)

ios::end

ios::current

Scanned with CamScanner

ifstream obj("abc.c");
obj.seekg(10, ios::beg);
skips 10 characters

obj.seekg(-15, ios::end);
last 15 characters

if(ch=='j')
obj.seekg(5, ios::cur);
skips 5 characters when termination come

class Dummy{
float i, j;
public:
Dummy(float a, float b){i=a; j=b;}
};
class Addition{
int x, y;
public:
Addition(int a, int b){x=a; y=b;}
int result(){return x+y;}
};

Be Positive...

Scanned with CamScanner

class A { public: void show() { cout << "Base"; } };
class B: public A { public: void show() { cout << "Derived"; } };
int main() { A *p; B ob2;

((B*)&p)->show(); // derived.
}

A ob1;
~~B *~~ B *p;
((A*)p)->show(); // Base

static cast → call by value
dynamic const cast → pointers

Note: ate is alternative of ios::app

Dt. 9f B+
Pg.

Ambiguity / Resolution in Multiple Inheritance

```
#include<iostream>
using namespace std;
class A
{ public:
   void display()
{ cout<<"Class A"<<endl;
}
};
class B
{public:
 void display()
{ cout<<"Class B"<<endl;
}};
class C: public A, public B
{void view()
 { display(); //↑
 }};
int main()
{  C c
 c.display(); //error (reference to display is ambiguous)
 return 0;
}
```

Solution →

```
void view()
{ A::display();
  B::display();
}
```

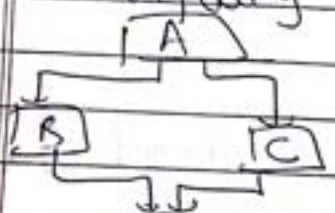
Be Positive...

Scanned with CamScanner

Ambiguity 2

```
class A
{public:
void display()
{cout<<"class A";
};
class B: public A
{public:
void display()
{cout<<"class B";
};
int main()
{ B b;
 b.display();   // display of B
 b.A::display(); // display of A
}
```

Ambiguity in Hybrid Inheritance



Both B & C inherit A they both have single copy of A. However D inherit both B & C
∴ D have 2 copies of A.
ambiguity.

2 ways to avoid
1. Scope Resolution operator
2. Virtual using virtual base class

1. Using scope resolution

```
class A
{ public:
    int a; };
class B: public A
{ public:
    int b; };
class C: public A
{ public:
    int c;
};
class D: public B, public C
{ public:
    int d; };
```

```
int main()
{ class D d;
  d.B::a=10
  d.C::a=100;

cout<<"A from B:"<<d.B::a;
cout<<"A from C:"<<d.C::a;
```

Output
A from B: 10
A from C: 100
Note: still there are 2 copies of A in D

2. Virtual base class → D has only one copy of A

```
class A
{ public:
    int a; };
class B: virtual public A
{ public:
    int b; };
class C: virtual public A
{ public:
    int c; };
class D: public B, public C
{ public:
    int d;
};
void main()
{ D d;
  d.a=10;
  d.a=100;
cout<<d.a<<endl //100
```

Be Positive...

Scanned with CamScanner